Week 6 Discussion

CSE 11 Winter 2026
November 4

Open Discussion Section 6 Interactive Groupwork on Gradescope

In today's gradescope submission, number 1-28, you will be filling in each with true/false

Subclasses

- Subclass extending only goes one way
  - Think of the Pict library
  - Pict c = new Circle(...);
  - **Question:** Why don't we write `Circle c = new Circle(...);`

```
Circle c = new Circle(...);
```

- The declared type of `c` is `Circle`.
- The actual object is a `Circle`.
- You can call methods defined in both `Circle` and `Pict` (subject to accessibility).

```
Pict c = new Circle(...);
```

- The declared type of `c` is `Pict`.
- The actual object is still a `Circle`.
- You can call methods available through `Pict`, but **NOT** methods exclusive to `Circle` without casting

Subclasses

1. A subclass can inherit methods and fields from its superclass.

2. In Java, a class can directly extend multiple classes.

3. The statement `Pict p = new Circle(10, "red");` is valid when Circle implements Pict.

4. If `Pict p = new Circle(10, "red");`, the actual object type is Pict.

Subclasses

1. A subclass can inherit methods and fields from its superclass.

TRUE: A subclass inherits accessible members of its superclass.

2. In Java, a class can directly extend multiple classes.

FALSE: Java supports only one direct superclass.

3. The statement `Pict p = new Circle(10, "red");` is valid when Circle implements Pict.

TRUE: A variable of an interface type can refer to an object implementing that interface.

4. If `Pict p = new Circle(10, "red");`, the actual object type is Pict.

FALSE: The declared variable type is Pict, but the actual object is a Circle.

Implements vs. Extends

quick reference

EXTENDS:

- **Inherits** from another class
- Represents an **IS-A** relationship
- Inherits accessible fields and methods
- Can **override** inherited methods
- A class can extend only ONE class

IMPLEMENTS:

- Implements an interface
- Represents a **CAN-DO** capability
- **MUST** provide implementations of required methods

BOTH:

- Support polymorphism
  - → ***Polymorphism*** *in Java means that one object can be treated as different types*
- Can be used together: `class Dog extends Animal implements Swimmable`
- Interfaces can also extend other interfaces

Implements vs. Extends

5. A class uses `implements` to declare that it fulfills an interface.

6. A class can implement more than one interface.

7. The statement `class Dog extends Animal implements Swimmable` is valid Java syntax, assuming the types are defined appropriately.

8. An interface can extend another interface.

Implements vs. Extends

5. A class uses `implements` to declare that it fulfills an interface.

TRUE: The class agrees to implement the interface's required methods.

6. A class can implement more than one interface.

TRUE: Java allows multiple interfaces separated by commas.

7. The statement `class Dog extends Animal implements Swimmable` is valid Java syntax, assuming the types are defined appropriately.

TRUE: A class may extend one class and implement one or more interfaces.

8. An interface can extend another interface.

TRUE: Interfaces can inherit from other interfaces using `extends`.

Overriding

- A subclass implements a method that is already defined in its superclass.
- You don't NEED the `@Override` annotation, it just provides metadata to the compiler!
  - Great for catching bugs, like misspellings of method when trying to override. Otherwise Java would simply treat it as a new method
- The method name must match.
- The parameter types must match.
- The return type must be the same *or* a compatible subtype.
- You cannot make the method less accessible (e.g., change public to private).
- You cannot override **`private`**, **`final`**, or **`static`** methods. **`static`** methods can be hidden, but not overridden.

Overriding

9. The `@Override` annotation is required for overriding a method.

10. An overriding method must have the same method name and parameter types as the inherited method.

11. A subclass can override a private method inherited from its superclass.

Overriding

9. The `@Override` annotation is required for overriding a method.

FALSE: The annotation is optional but helps the compiler catch mistakes.

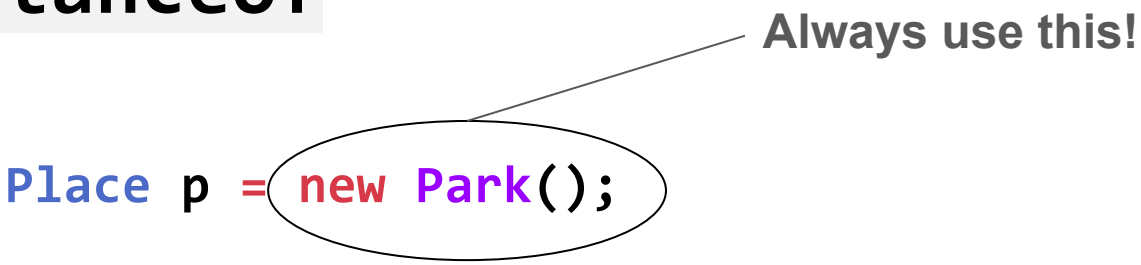
10. An overriding method must have the same method name and parameter types as the inherited method.

TRUE: The method signature must match.

11. A subclass can override a private method inherited from its superclass.

FALSE: Private methods are not inherited and cannot be overridden.

instanceof



- Even though **p** is declared as a `Place`, the actual object is a `Park`.
- An object can be an instance of multiple types because of inheritance and interfaces.
- It lets us check an object's type before using subclass-specific methods.
- A subclass object is also an instance of its superclass
- An object that implements an interface is also an instance of the interface type

`instanceof`

12. The `instanceof` operator returns a boolean.

13. If `Animal a = new Dog();`, then `a instanceof Dog` evaluates to false because a is declared as Animal.

14. An object can be an instance of both a class and an interface.

15. The expression `null instanceof Object` evaluates to true.

`instanceof`

12. The `instanceof` operator returns a boolean.

TRUE: It evaluates to true or false.

13. If `Animal a = new Dog();`, then `a instanceof Dog` evaluates to false because a is declared as Animal.

FALSE: `instanceof` checks the actual object, which is a Dog.

14. An object can be an instance of both a class and an interface.

TRUE: An object can be an instance of its class and interfaces implemented by that class.

15. The expression `null instanceof Object` evaluates to true.

FALSE: `null instanceof Object` is always false.

Overriding the `equals()` method

```
public boolean equals(Object other) {

  if (this == other) { return true; }                    // 1. Check for reference equality.

  if (!(other instanceof Location)){ return false;}      // 2. Check instanceof

  Location l = (Location)other;                          // 3. Cast and check fields
  return lat == l.lat && lng == l.lng;
 }
}
```

Overriding the `equals()` method

16. The `==` operator compares whether two object references refer to the same object.

17. The `equals()` method always compares the contents of objects, even when it has not been overridden.

18. A properly implemented `equals()` method should return false when the argument is null.

19. When overriding `equals()`, the method should have the signature `public boolean equals(Object other)`.

Overriding the `equals()` method

16. The `==` operator compares whether two object references refer to the same object.

TRUE: For reference types, `==` tests reference identity.

17. The `equals()` method always compares the contents of objects, even when it has not been overridden.

FALSE: Object's default `equals()` implementation compares reference identity.

18. A properly implemented `equals()` method should return false when the argument is null.

TRUE: An object is never equal to null.

19. When overriding `equals()`, the method should have the signature `public boolean equals(Object other)`.

TRUE: Using Object as the parameter type overrides `Object.equals()`.

Abstract classes

- Concrete classes can be instantiated using `new`
  - Abstract classes get a compile error when trying to use `new`
- An abstract class is meant to serve as a parent class for other classes
- Abstract classes can contain **both** abstract methods (no implementation) and concrete methods (with implementation)
- An abstract class describes what something is, while sharing state and behavior.
- An interface, on the other hand, describes what something can do, or a contract it fulfills.

Abstract classes

20. You can instantiate an abstract class using new.

21. An abstract class can contain both abstract and concrete methods.

22. A concrete subclass must implement all inherited abstract methods that remain unimplemented.

23. Abstract classes cannot contain fields.

Abstract classes

20. You can instantiate an abstract class using new.

FALSE: Abstract classes cannot be directly instantiated.

21. An abstract class can contain both abstract and concrete methods.

TRUE: Abstract methods lack bodies, while concrete methods provide implementations.

22. A concrete subclass must implement all inherited abstract methods that remain unimplemented.

TRUE: Otherwise the subclass must itself be declared abstract.

23. Abstract classes cannot contain fields.

FALSE: Abstract classes can contain fields and constructors.

Sharing vs. Copying

SHARING

- Two variables refer to the same object in memory.
- Changing a shared mutable object through one reference affects what the other reference sees.

COPYING

- Two variables refer to different objects that may contain the same data.
- Changing a copied object does not affect the original
- Using = with objects does NOT copy the object, it copies the reference (aka sharing)

Sharing vs. Copying

24. Assigning one object reference variable to another automatically creates a copy of the object.

25. Two variables can refer to the exact same object in memory.

26. If two variables refer to the same mutable object, changes made through one reference are visible through the other.

27. Creating a shallow copy guarantees that all nested mutable objects are independent.

Sharing vs. Copying

24. Assigning one object reference variable to another automatically creates a copy of the object.

FALSE: It copies the reference, so both variables point to the same object.

25. Two variables can refer to the exact same object in memory.

TRUE: This is called sharing or aliasing.

26. If two variables refer to the same mutable object, changes made through one reference are visible through the other.

TRUE: Both references access the same underlying object.

27. Creating a shallow copy guarantees that all nested mutable objects are independent.

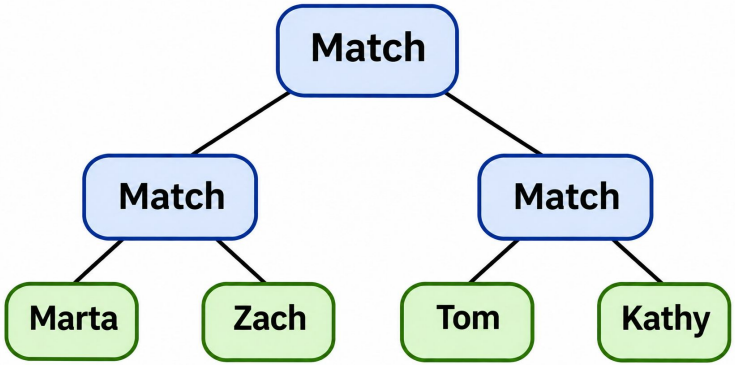
FALSE: A shallow copy can still share references to nested objects.

PA3 Overview

- Expressions
  - Design and implement a program that represents mathematical expressions as data: numbers, variables, sums, differences, products, and quotients, nested to any depth.
  - Render, evaluate, and substitute them

Question 1: Tournament bracket

- Every match has two participants, and each participant can either be a player or the winner of another match.
- What interfaces and records would you create?
  Write the interface and record headers you would use.
- How would you represent the bracket on the right?
  Write out the code to create this tournament.



PA3 Overview

- Chess
  - Create interfaces and records based on the guidelines, draw a chess board and mark where a piece can move

Question 1: Which of these code blocks will compile/run?

Using these interfaces and records

```
interface Piece {
   MoveList getMoves(int row, int col, Board board);
}

record Rook() implements Piece { ... }
record Queen() implements Piece { ... }
...
```

1. `Piece p = new Queen();`
2. `Queen q = new Piece();`
3. `Piece p = new Rook();`
4. `Queen q = new Rook();`
5.
```
Piece[] pieces = new Piece[3];
pieces[0] = new Queen();
pieces[1] = new Rook();
```
6.
```
Queen[] queens = new Queen[3];
queens[0] = new Queen();
queens[1] = new Rook();
```
7.
```
Queen[] queens = new Queen[2];
Piece p = new Queen();
queens[0] = p;
```
8.
```
Piece[] pieces = new Queen[2];
pieces[0] = new Rook();
```

PA3 Overview

- Chess
  - Create interfaces and records based on the guidelines, draw a chess board and mark where a piece can move

Question 1: Which of these code blocks will compile/run?

Using these interfaces and records

```
interface Piece {
   MoveList getMoves(int row, int col, Board board);
}

record Rook() implements Piece { ... }
record Queen() implements Piece { ... }
...
```

1. `Piece p = new Queen();`
2. `Queen q = new Piece();`
3. `Piece p = new Rook();`
4. `Queen q = new Rook();`
5.
```
Piece[] pieces = new Piece[3];
pieces[0] = new Queen();
pieces[1] = new Rook();
```
6.
```
Queen[] queens = new Queen[3];
queens[0] = new Queen();
queens[1] = new Rook();
```
7.
```
Queen[] queens = new Queen[2];
Piece p = new Queen();
queens[0] = p;
```
8.
```
Piece[] pieces = new Queen[2];
pieces[0] = new Rook();
```

TODO

- **PA03 is due THIS FRIDAY Nov 6**

Thank You!
Any Questions?